Lab 7: Fuzzing for Input Validation Bugs

Student	**Nikhil Reddy** · SRN PES1UG24CS583
Application	`lab7/tennis_scoring.py` (parse_score, game_lead, set_spread)
My repo (fork)	github.com/nikhilreddynani1-beep/L6_L7_34_tennis_scoring/tree/main/lab7
Assigned repo	github.com/SETAPESU26/L6_L7_34_tennis_scoring/tree/main/lab7
Tools	Python 3.13 · Hypothesis 6.170 · pytest 9.1

Summary

Run	Buggy code	Fixed code
`python tennis_scoring.py` with invalid input	Crashes with a traceback	Prints "Invalid input..." and continues
`pytest test_tennis_scoring.py` (8 Hypothesis tests)	**5 failed**, 3 passed	**8 passed**

Bugs found by fuzzing

Smallest failing inputs reported by Hypothesis on the buggy code:

Function	Input	Crash	Cause
`parse_score`	`''`	ValueError: not enough values to unpack	No '-' in the text, so split gives one part
`parse_score`	`'-'`	ValueError: invalid literal for int() ''	Empty number on one side of '-'
`parse_score`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string
`parse_score`	`'1' × 4301 + '-1'`	ValueError: exceeds the limit (4300 digits)	int() refuses very long digit strings
`parse_score`	`'-1-0'`	ValueError: too many values to unpack	Negative number adds an extra '-'
`game_lead`	`'-'`	KeyError: ''	Empty side is not a tennis point name
`game_lead`	`''`	ValueError: not enough values to unpack	No '-' in the text
`game_lead`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string
`set_spread`	`''`	ValueError: invalid literal for int() ''	Empty CSV / empty game count
`set_spread`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string

Fixes applied

Where	Fix
All three	Return `None` for input that is not a string, instead of calling `.split()` on it.
`parse_score`, `game_lead`	Split on '-' and require exactly two parts; strip spaces around each part.
`parse_score`, `set_spread`	Accept a count only if it is 1–4 plain ASCII digits (regex `[0-9]{1,4}`). This rejects empty parts, signs, decimals, Unicode digits such as '²', and numbers too long for

	`int()`.
`game_lead`	Look the two sides up only if both are valid point names (0, 15, 30, 40).
`set_spread`	Reject the whole CSV if any game count is invalid, so `max()`/`min()` never see an empty or bad list.
`main`	Print a clear 'Invalid input' message when a function returns `None`, and stop cleanly if input ends (EOF).

Every function now returns `None` for input it cannot interpret, instead of raising an exception.

Tests written in `test_tennis_scoring.py`

- **No-crash tests** for all three functions, using `st.text() | st.none()` plus score-like text (digits, '-', ',', '+', spaces) and 500 examples each. Each test also checks the result is either `None` or a valid value: a pair of non-negative ints, a lead between −3 and +3, or a non-negative spread.
- **Huge number:** numbers of 4301–6000 digits in `parse_score`.
- **Correct results for valid input:** `parse_score("a-b") == (a, b)`; `game_lead` matches the point difference and flips sign when the players are swapped; `set_spread` equals max − min.
- **Rejects negative points** in `parse_score`.

Step 2: run the buggy application

A valid run works, but entering `3` (no '-') crashes the program.

TERMINAL · lab7 — Step 2: buggy tennis_scoring.py

```
$ python3 tennis_scoring.py
==== Running parse_score ====
Enter points won as A-B (e.g. 3-2): 3-2
Points A and B: (3, 2)

==== Running game_lead ====
Enter a game score (e.g. 40-30): 40-30
Lead in Points: 1

==== Running set_spread ====
Enter games won per set as CSV (e.g. 6,4,7): 6,4,7
Spread: 3

$ python3 tennis_scoring.py
==== Running parse_score ====
Enter points won as A-B (e.g. 3-2): 3
Traceback (most recent call last):
  File "/home/claude/l7/work/buggy/tennis_scoring.py", line 31, in <module>
    main()
    ~~~~^^
  File "/home/claude/l7/work/buggy/tennis_scoring.py", line 17, in main
    result = parse_score(user_input)
  File "/home/claude/l7/work/buggy/tennis_scoring.py", line 2, in parse_score
    a, b = text.split('-')
    ^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Command: `python tennis_scoring.py`

Step 4: fuzz tests on the buggy code (failing)

TERMINAL · lab7 — Step 4: fuzz tests on buggy code (top)

```
$ python3 -m pytest test_tennis_scoring.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
hypothesis profile 'default'
rootdir: /home/claude/l7/work/buggy
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.15
collecting ... collected 8 items

test_tennis_scoring.py::test_parse_score_no_crash FAILED                 [ 12%]
test_tennis_scoring.py::test_parse_score_huge_number_no_crash FAILED     [ 25%]
test_tennis_scoring.py::test_parse_score_valid_round_trip PASSED         [ 37%]
test_tennis_scoring.py::test_parse_score_rejects_negative_points FAILED  [ 50%]
test_tennis_scoring.py::test_game_lead_no_crash FAILED                   [ 62%]
test_tennis_scoring.py::test_game_lead_valid_scores PASSED               [ 75%]
test_tennis_scoring.py::test_set_spread_no_crash FAILED                  [ 87%]
test_tennis_scoring.py::test_set_spread_valid_sets PASSED                [100%]
```

Command: `python -m pytest test_tennis_scoring.py -v`: 5 of 8 tests fail.

TERMINAL · lab7 — Step 4: fuzz tests on buggy code (end of output)

```
During handling of the above exception, another exception occurred:

    @settings(max_examples=500)
>   @given(any_input)
                ^^^

test_tennis_scoring.py:73:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = ''

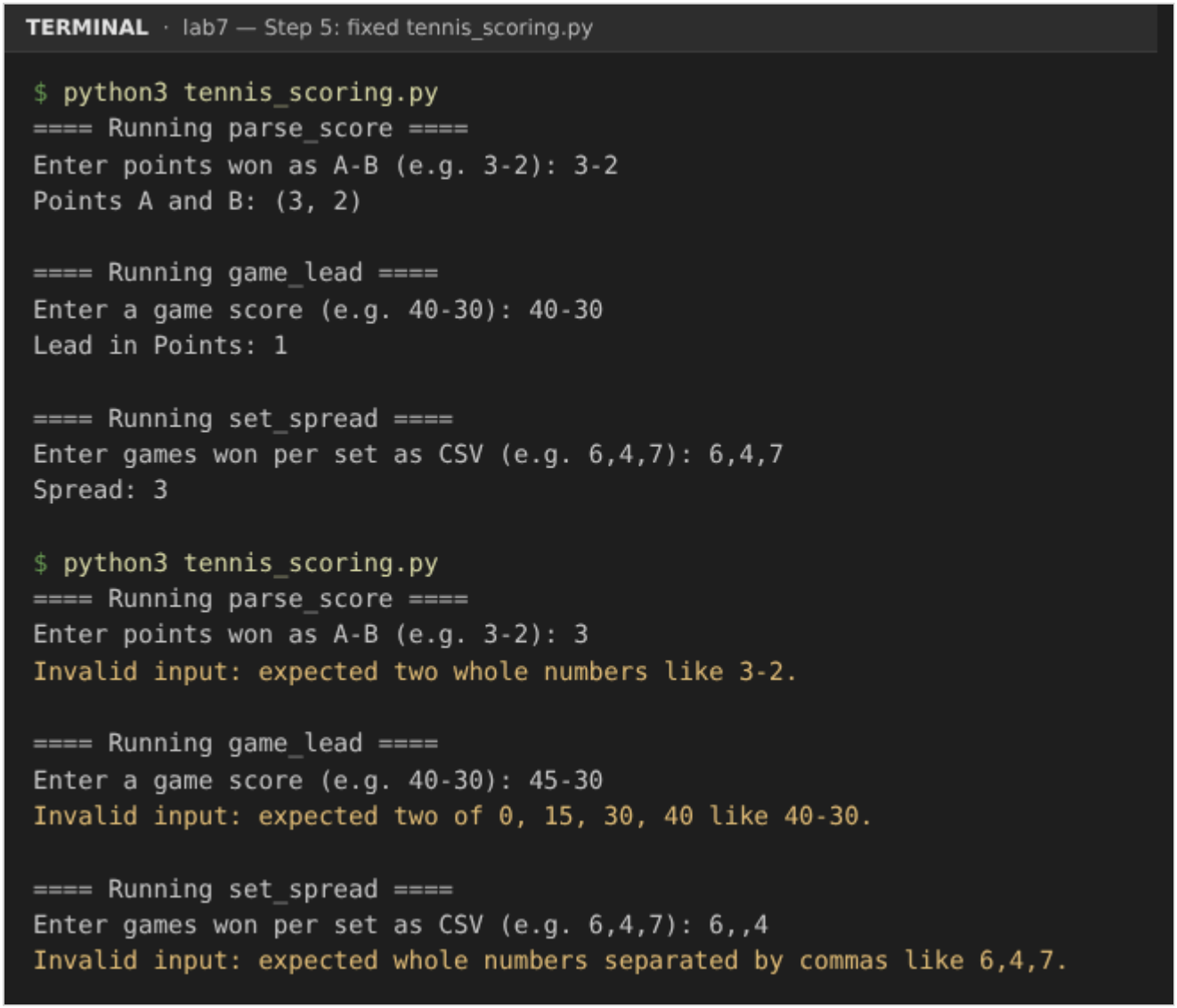
    @settings(max_examples=500)
    @given(any_input)
    def test_set_spread_no_crash(s):
        try:
            result = set_spread(s)
        except Exception as e:
>           assert False, f"set_spread crashed with: {e!r}"
E           AssertionError: set_spread crashed with: ValueError("invalid literal for int() with base 10: ''")
E           assert False
E           Failing test case: test_set_spread_no_crash(
E               s='',
E           )
E           Explanation:
E               These lines were always and only run by failing test cases:
E                   /home/claude/l7/work/buggy/test_tennis_scoring.py:77

test_tennis_scoring.py:78: AssertionError
=========================== short test summary info ============================
FAILED test_tennis_scoring.py::test_parse_score_no_crash - ExceptionGroup: Hy...
FAILED test_tennis_scoring.py::test_parse_score_huge_number_no_crash - Assert...
FAILED test_tennis_scoring.py::test_parse_score_rejects_negative_points - Val...
FAILED test_tennis_scoring.py::test_game_lead_no_crash - ExceptionGroup: Hypo...
FAILED test_tennis_scoring.py::test_set_spread_no_crash - AssertionError: set...
======================== 5 failed, 3 passed in 2.57s ===========================
```

End of the same output: Hypothesis's smallest failing case for `set_spread` (`s=''`) and the summary.

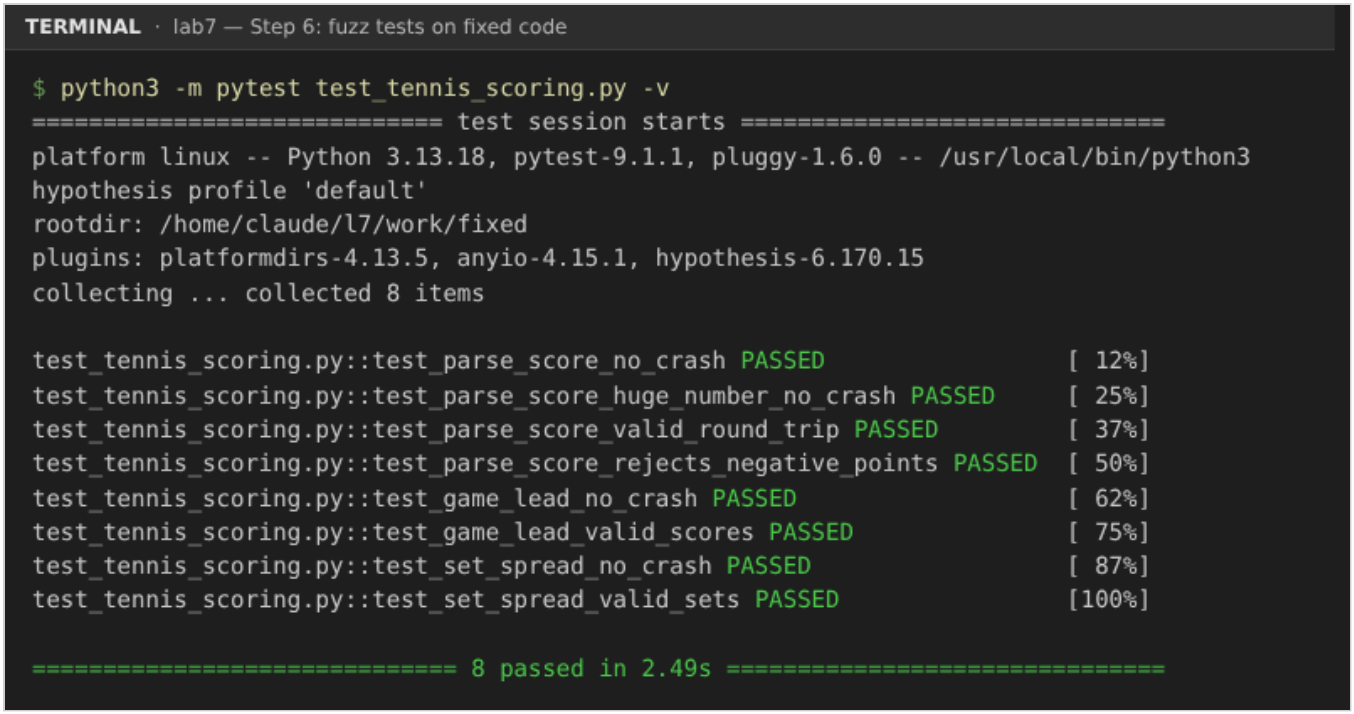
Step 5: run the fixed application

The same valid run gives the same results; invalid input now prints a message instead of crashing.



Command: `python tennis_scoring.py`

Step 6: fuzz tests on the fixed code (passing)



Command: `python -m pytest test_tennis_scoring.py -v`: all 8 tests pass.